

ОТЧЕТ

Многослойный персептрон

Выполнил:

студент гр. 09-321

Калимуллина Г.И.

Содержание

1	Введение	3
2	Описание датасета	4
3	Разведочный анализ	5
4	Теоретическая часть	9
5	Реализация	13
6	Результаты	16
7	Заключение	18

Введение

Машинное обучение, и в частности искусственные нейронные сети, широко применяются в современной медицине для задач диагностики. Одной из классических задач является бинарная классификация опухолей молочной железы на доброкачественные и злокачественные по результатам тонкоигльной биопсии. Ранняя и точная диагностика критически важна для выбора стратегии лечения.

В данной работе рассматривается задача бинарной классификации: по вектору из 30 числовых признаков, описывающих характеристики ядра клетки, полученных при тонкоигльной аспирационной пункционной биопсии, необходимо предсказать класс опухоли — злокачественная (M, malignant) или доброкачественная (B, benign). Цель проекта — реализовать многослойный персептрон с нуля, без использования готовых ML-библиотек (Keras, PyTorch, TensorFlow), и обучить его на предоставленном наборе данных.

Задачи проекта:

- провести разведочный анализ данных;
- реализовать полносвязный слой с прямым и обратным проходом;
- реализовать функции активации (сигмоида), softmax и бинарную кросс-энтропию;
- реализовать цикл обучения с использованием мини-батчeveго градиентного спуска;
- обучить модель и оценить её качество на отложенной выборке.

Проект реализован на языке Python. Для матричных операций использован NumPy, для работы с данными — pandas, для визуализации — matplotlib.

Описание датасета

В работе использован набор данных, предоставленный в рамках задания. Признаки описывают характеристики ядра клетки молочной железы, полученные методом тонкоигольной аспирационной пункционной биопсии.

Набор данных представлен в файле `data.csv` в формате CSV и содержит 569 наблюдений. Каждое наблюдение описано:

- идентификатором (`id`),
- 30 числовыми признаками, характеризующими ядро клетки,
- меткой класса (`diagnosis`).

Целевая переменная `diagnosis` принимает два значения:

- М (`malignant`) — злокачественная опухоль, 212 наблюдений (37.3%);
- В (`benign`) — доброкачественная опухоль, 357 наблюдений (62.7%).

Классы несбалансированы (соотношение примерно 1.68:1), однако дисбаланс умеренный и не требует применения специальных техник балансировки. Пропусков в данных нет. Все признаки — числовые, что позволяет применять их напрямую после нормализации.

Разведочный анализ

Перед обучением модели был проведён анализ данных. Его цель — понять структуру данных, выявить закономерности и подготовить основу для дальнейшей обработки.

На рисунке 1 представлено распределение наблюдений по классам. Класс В (доброкачественная опухоль) содержит 357 наблюдений (62.7%), класс М (злокачественная) — 212 наблюдений (37.3%). Соотношение составляет примерно 1.68:1, то есть дисбаланс умеренный.

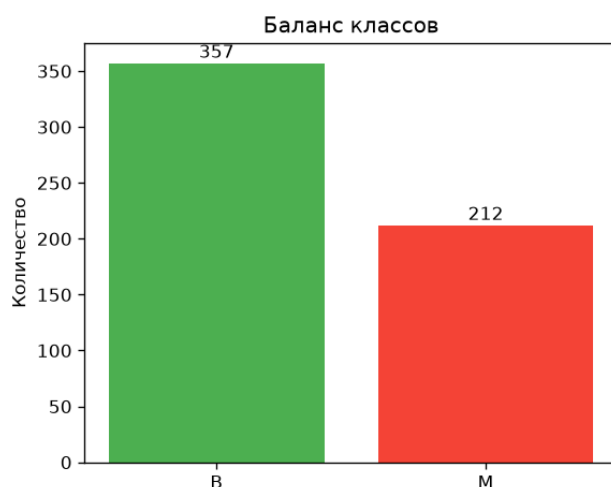


Рис. 1 – Распределение наблюдений по классам

Умеренный дисбаланс означает, что метрика ассигасу сама по себе не является полностью информативной: модель, всегда предсказывающая класс В, даст точность 62.7%. Поэтому при оценке качества модели дополнительно анализируются precision, recall и F1-мера для класса М.

На рисунке 2 приведены гистограммы первых шести признаков в разрезе классов. Видно, что одни признаки хорошо разделяют классы, а другие — слабо.

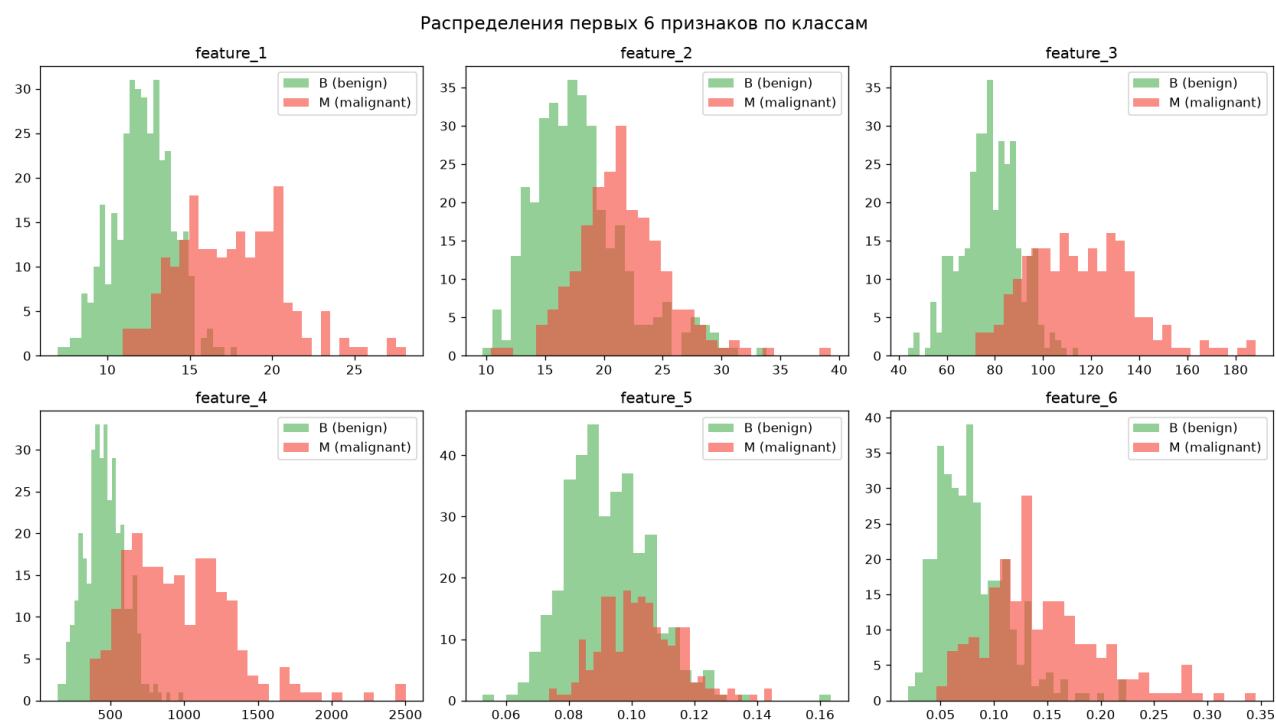


Рис. 2 – Распределения первых шести признаков по классам

Анализ гистограмм показывает, что часть признаков имеет слабое перекрытие распределений классов, то есть является информативной для классификации. Другие признаки, напротив, имеют почти полностью совпадающие распределения и слабо влияют на решение задачи.

На рисунке 3 представлена матрица корреляций признаков. Обнаружены группы сильно коррелирующих признаков.

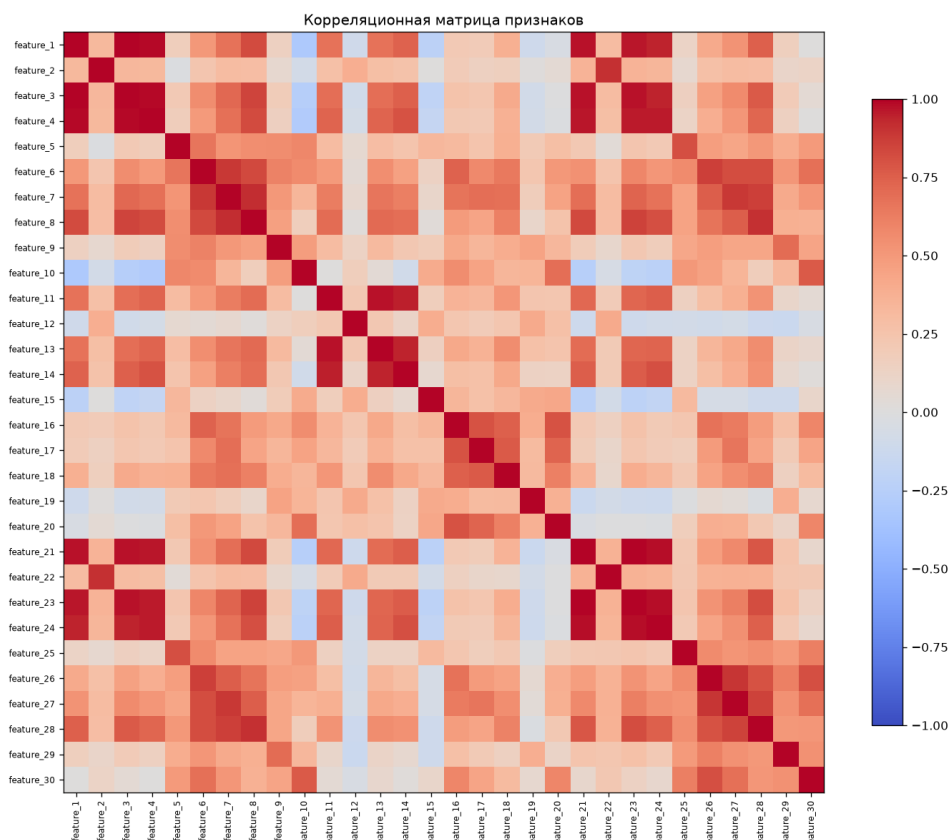


Рис. 3 – Корреляционная матрица признаков

Высокая корреляция означает, что часть признаков дублирует информацию друг друга. Для многослойного персептрона это не является проблемой — сеть способна работать с избыточными признаками, — однако важно учитывать это при интерпретации результатов.

Анализ описательных статистик (минимум, максимум, среднее, стандартное отклонение) показал, что признаки имеют существенно разные масштабы: значения варьируются от порядка 10^{-3} до порядка 10^3 . Без нормализации это привело бы к доминированию крупномасштабных признаков в градиентном спуске. Поэтому на этапе предобработки была применена стандартизация (z-нормализация).

По результатам разведочного анализа можно сделать следующие выводы:

- данные не содержат пропусков, все признаки — числовые;
- классы несбалансированы (примерно 1.68:1), однако дисбаланс умеренный и не требует специальных техник балансировки;
- часть признаков хорошо разделяет классы, часть — слабо;

- признаки имеют разные масштабы, что делает необходимой нормализацию перед обучением;
- обнаружены сильно коррелирующие группы признаков;
- признак `id` не несёт информации о классе и был удалён на этапе предобработки.

Теоретическая часть

Многослойный персептрон (Multi-Layer Perceptron, MLP) — это нейронная сеть прямого распространения, состоящая из нескольких последовательно соединённых слоёв нейронов. Каждый нейрон слоя связан со всеми нейронами предыдущего слоя, поэтому такие слои называются полносвязными (Dense).

Архитектура MLP включает:

- входной слой — принимает вектор признаков объекта;
- скрытые слои — промежуточные полносвязные слои с нелинейной функцией активации;
- выходной слой — формирует выход сети, в задачах классификации — вероятности классов.

В данной работе используется архитектура с тремя скрытыми слоями по 24 нейрона и выходным слоем из 2 нейронов:

$$30 \rightarrow 24 \rightarrow 24 \rightarrow 24 \rightarrow 2.$$

Нелинейные функции активации в скрытых слоях позволяют сети аппроксимировать сложные нелинейные зависимости. Без них вся сеть свелась бы к одному линейному преобразованию.

Пусть на вход слоя подаётся матрица X размера $(B \times n)$, где B — размер мини-батча, n — число входных признаков. Слой характеризуется матрицей весов W размера $(n \times m)$ и вектором смещений b размера (m) , где m — число нейронов слоя.

Линейное преобразование:

$$Z = XW + b.$$

Применение функции активации σ :

$$A = \sigma(Z).$$

Матрица A размера $(B \times m)$ — выход слоя, который подаётся на вход следующему слою.

В скрытых слоях используется логистическая функция (сигмоида):

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Свойства сигмоиды:

- область значений — интервал $(0, 1)$;
- $\sigma(0) = 0.5$;
- при $z \rightarrow +\infty$ значение стремится к 1, при $z \rightarrow -\infty$ — к 0;
- функция дифференцируема на всей числовой оси.

Производная сигмоиды выражается через саму функцию:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)).$$

Это свойство важно для обратного распространения: имея значение $A = \sigma(Z)$, мы можем вычислить производную без повторного вычисления сигмоиды.

На выходном слое применяется функция softmax, которая преобразует вектор «сырых» значений z в вероятностное распределение:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Свойства:

- все компоненты положительны;
- сумма компонент равна единице;
- чем больше z_i относительно остальных, тем выше вероятность класса i .

Для численной устойчивости перед возведением в экспоненту вычитается максимум вектора:

$$\text{softmax}(z)_i = \frac{e^{z_i - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}}.$$

Это преобразование не меняет результат математически, но предотвращает переполнение при больших значениях z .

Для задачи бинарной классификации используется бинарная кросс-энтропия. Пусть $\hat{y}_i$ — предсказанная вероятность класса 1 для i -го объекта, $y_i \in \{0, 1\}$ — истинная метка, N — число объектов в батче. Тогда

$$L = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

Функция штрафует модель тем сильнее, чем увереннее она ошибается:

- при $y_i = 1$ вклад равен $-\log \hat{y}_i$;
- при $y_i = 0$ вклад равен $-\log(1 - \hat{y}_i)$.

Обучение сети основано на методе обратного распространения ошибки (backpropagation). Идея метода — применить правило цепочки для вычисления производных функции потерь по всем параметрам сети.

Пусть на выходе слоя получен градиент $\partial L / \partial A$, пришедший от следующего слоя. Тогда:

1. Градиент по линейному выходу:

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \cdot \sigma'(Z).$$

2. Градиент по весам:

$$\frac{\partial L}{\partial W} = X^T \frac{\partial L}{\partial Z}.$$

3. Градиент по смещениям:

$$\frac{\partial L}{\partial b} = \sum_{i=1}^B \frac{\partial L}{\partial Z_i}.$$

4. Градиент для предыдущего слоя:

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} W^T.$$

Формула (4) позволяет передать градиент на предыдущий слой, последовательно применяя те же вычисления ко всем слоям сети.

Обновление параметров выполняется методом стохастического градиентного спуска. На каждом шаге параметры сдвигаются в направлении, противоположном градиенту:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b},$$

где η — скорость обучения (learning rate).

Обучение проводится по эпохам. В начале каждой эпохи данные перемешиваются и разбиваются на мини-батчи размера B . Для каждого батча выполняются:

- прямой проход (forward);
- вычисление функции потерь;
- обратный проход (backward);
- обновление весов.

Использование мини-батчей даёт компромисс между точностью оценки градиента и частотой обновления параметров: с одной стороны, градиент усредняется по нескольким объектам (снижение дисперсии), с другой — обновления происходят достаточно часто, что ускоряет сходимость на малых наборах данных.

Реализация

Все программы реализованы на языке Python с использованием только базовых библиотек: NumPy (матричные операции), pandas (работа с данными) и matplotlib (визуализация).

Проект состоит из следующих модулей:

- `split_data.py` — разделение исходного набора данных на обучающую и валидационную выборки, нормализация признаков;
- `layers.py` — реализация полносвязного слоя `DenseLayer` с прямым и обратным проходом;
- `losses.py` — функции `softmax`, бинарной кросс-энтропии и её градиента;
- `train.py` — класс `NeuralNetwork` и цикл обучения с мини-батчевым градиентным спуском;
- `predict.py` — загрузка обученной модели и оценка её качества на валидационной выборке.

На этапе предобработки (`split_data.py`) выполняются следующие шаги:

- загрузка исходного файла `data.csv`;
- удаление идентификатора `id`, не несущего информации о классе;
- кодирование целевой переменной: $M \rightarrow 1, B \rightarrow 0$;
- разделение выборки на обучающую (80%) и валидационную (20%) с фиксированным `seed` для воспроизводимости;
- стандартизация признаков (z-нормализация):

$$x_{\text{norm}} = \frac{x - \mu}{\sigma},$$

где μ и σ вычисляются только на обучающей выборке для избежания утечки данных (data leakage);

- сохранение нормализованных выборок в `data_training.csv` и `data_valid.csv`, а параметров нормализации — в `normalization_params.csv`.

Класс `DenseLayer` (`layers.py`) хранит матрицу весов W , вектор смещений b и тип активации. Инициализация весов выполняется методом `He`:

$$W_{ij} \sim \mathcal{U} \left(-\sqrt{\frac{6}{n}}, \sqrt{\frac{6}{n}} \right),$$

где n — число входных признаков слоя. Такой масштаб весов обеспечивает стабильное обучение при использовании сигмоиды.

Класс реализует три метода:

- `forward(x)` — прямой проход: вычисление $Z = XW + b$ и $A = \sigma(Z)$;
- `backward(dL_da)` — обратный проход: вычисление градиентов $\partial L / \partial W$, $\partial L / \partial b$ и $\partial L / \partial X$;
- `update(lr)` — обновление весов: $W \leftarrow W - \eta \partial L / \partial W$.

Модуль `losses.py` содержит три функции:

- `softmax(z)` — преобразование «сырых» выходов в вероятности с вычитанием максимума для численной устойчивости;
- `cross_entropy_loss(y_pred, y_true)` — вычисление средней бинарной кросс-энтропии по батчу. Для защиты от $\log(0)$ применяется ограничение снизу: значения вероятностей зажимаются в диапазон $[10^{-12}, 1]$;
- `cross_entropy_grad(y_pred, y_true)` — вычисление градиента на выходе сети. Благодаря совместному применению `softmax` и кросс-энтропии градиент имеет простую форму:

$$\frac{\partial L}{\partial Z_{\text{out}}} = \frac{1}{N}(\hat{Y} - Y),$$

где $\hat{Y}$ — предсказанные вероятности, Y — one-hot представление истинных меток.

В файле `train.py` реализованы:

- класс `NeuralNetwork`, инкапсулирующий список слоёв и выполняющий прямой и обратный проход по всей сети последовательно;
- функция `train_model`, реализующая цикл обучения. На каждой эпохе данные перемешиваются, разбиваются на мини-батчи размера 8, и для каждого батча выполняются прямой проход, вычисление градиента и обновление весов;
- функции `compute_loss` и `compute_acc` для оценки метрик на обучающей и валидационной выборках.

Архитектура сети:

$$30 \rightarrow 24 \rightarrow 24 \rightarrow 24 \rightarrow 2.$$

Гиперпараметры обучения:

- число эпох: 70;
- размер мини-батча: 8;
- скорость обучения: $\eta = 0.0314$;
- seed: 42 (для воспроизводимости).

По завершении обучения веса всех слоёв сохраняются в файл `saved_model.npz` с помощью `numpy.savez`.

Файл `predict.py` выполняет:

- загрузку валидационной выборки и весов обученной модели;
- создание сети той же архитектуры и подстановку сохранённых весов;
- прямой проход по валидационной выборке;
- вычисление бинарной кросс-энтропии и ассурасы;
- построение матрицы ошибок (confusion matrix) и вычисление precision, recall и F1-меры для класса М.

Матрица ошибок позволяет детально проанализировать характер ошибок модели — в частности, различить ложноположительные и ложноотрицательные срабатывания, что важно в медицинских задачах.

Результаты

На рисунке 4 представлены кривые обучения: зависимость функции потерь и точности от номера эпохи для обучающей и валидационной выборок.

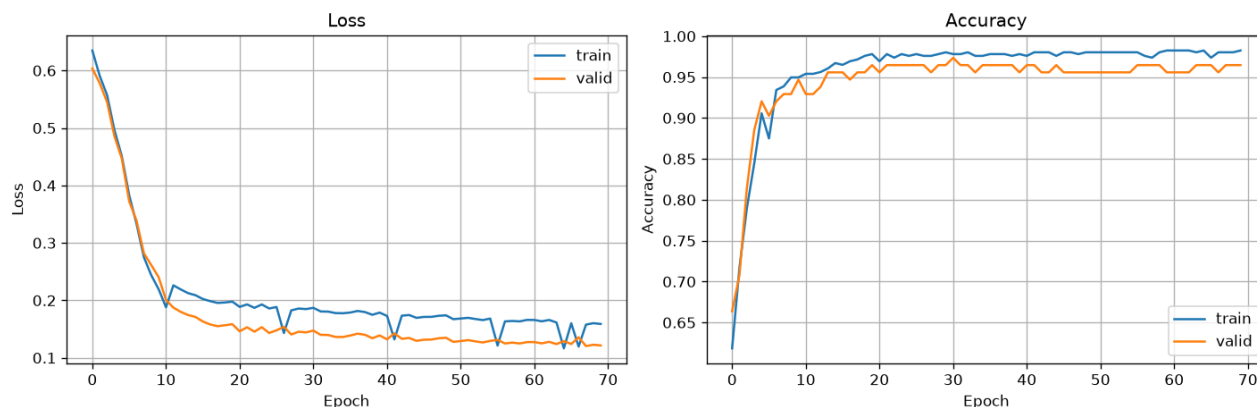


Рис. 4 – Кривые обучения: функция потерь (слева) и точность (справа)

Из графиков видно, что обучение сходится за первые 15–20 эпох, после чего кривые выходят на плато. Функция потерь на обучающей выборке снижается с 0.63 до 0.16, на валидационной — с 0.60 до 0.12. Точность на обучающей выборке растёт с 0.62 до 0.98, на валидационной — с 0.66 до 0.96.

Кривые на обучающей и валидационной выборках идут близко друг к другу на всём протяжении обучения. Разрыв между ними невелик (около 2%), что говорит об отсутствии переобучения. Небольшие колебания на обучающей кривой объясняются стохастичностью мини-батчевого градиентного спуска: веса обновляются много раз за эпоху, поэтому значение функции потерь, замеренное в конце эпохи, слегка варьируется.

После завершения обучения модель была оценена на валидационной выборке, содержащей 113 наблюдений (75 класса В и 38 класса М). В таблице 1 приведены основные метрики.

Таблица 1 – Метрики качества на валидационной выборке

Метрика	Значение
Бинарная кросс-энтропия	0.1216
Accurasy	0.9646
Precision (класс М)	1.0000
Recall (класс М)	0.8947
F1-мера (класс М)	0.9444

Точность 96.46% означает, что модель правильно классифицировала 109 из 113 наблюдений.

Precision = 1.000 для класса М означает, что все случаи, предсказанные моделью как злокачественные, оказались истинно злокачественными — ложных срабатываний нет.

Recall = 0.895 означает, что модель обнаружила 34 из 38 истинно злокачественных наблюдений. Оставшиеся 4 были ошибочно отнесены к доброкачественным.

В таблице 2 представлена матрица ошибок — распределение истинных и предсказанных классов.

Таблица 2 – Матрица ошибок

	Предсказано В	Предсказано М
Истинно В	75 (TN)	0 (FP)
Истинно М	4 (FN)	34 (TP)

Разбор:

- $TN = 75$ — все доброкачественные опухоли верно опознаны;
- $FP = 0$ — ложных срабатываний нет;
- $FN = 4$ — четыре злокачественные опухоли были пропущены;
- $TP = 34$ — большинство злокачественных опухолей обнаружены верно.

С точки зрения медицинской практики результаты показывают осторожный характер модели: она избегает ложных тревог ($FP = 0$), но при этом допускает пропуски ($FN = 4$). В задаче диагностики рака предпочтительнее была бы обратная стратегия — допускать ложные срабатывания в пользу более высокого Recall, поскольку пропущенная злокачественная опухоль опаснее ложной тревоги. В дальнейшем Recall можно повысить, например, изменением порога классификации или введением весов классов в функцию потерь. Однако в рамках текущего задания модель показала высокое качество классификации.

Заключение

В ходе работы был реализован многослойный персептрон с нуля на языке Python без использования готовых библиотек глубокого обучения. Все компоненты нейронной сети — полносвязный слой, функции активации, функция потерь, прямой и обратный проход, алгоритм градиентного спуска — реализованы самостоятельно на основе библиотеки NumPy.

Были выполнены следующие задачи:

- проведён разведочный анализ данных: изучены распределения признаков, баланс классов, корреляции и масштабы;
- выполнена предобработка данных: кодирование целевой переменной, разделение на обучающую и валидационную выборки, стандартизация признаков;
- реализован полносвязный слой с прямым и обратным проходом, включая вычисление градиентов по правилу цепочки;
- реализованы функции активации (сигмоида), softmax и бинарная кросс-энтропия;
- реализован цикл обучения с использованием мини-батчeveго градиентного спуска;
- обучена модель архитектуры $30 \rightarrow 24 \rightarrow 24 \rightarrow 24 \rightarrow 2$;
- проведена оценка качества модели на отложенной выборке.

Обученная модель достигла точности 96.46% на валидационной выборке, что соответствует высокому уровню качества для данной задачи. Кривые обучения показали отсутствие переобучения и сходимость за первые 15–20 эпох.

Цель работы достигнута: многослойный персептрон реализован, обучен и успешно применён к задаче бинарной классификации.